Doubly Linked List – Insert At End And Time Complexity.

Program :

```
void insertAtEnd(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = nullptr;

    if (head == nullptr)
    {
        newNode->prev = nullptr;
        head = newNode;
        cout << data << " inserted at the end." << endl;
        return;
    }
    Node *temp = head;
    while (temp->next != nullptr)
    {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
    cout << data << " inserted at the end." << endl;
}

…
        case 2:
            cout << "Enter data to insert: ";
            cin >> data;
            insertAtEnd(data);
            break;
```

Program Analysis:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

→ ***The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.***

→ ***The size of a pointer (newNode) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.***

→ ***The size of the Node structure itself is determined by the sum of the sizes of its members***:

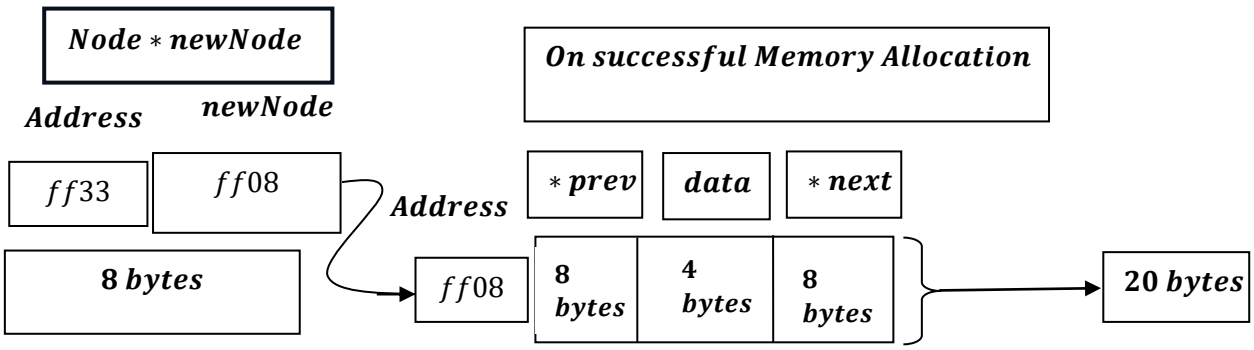
→ ***data(an integer) is typically 4 bytes.***

→ ***next(a pointer to another Node) is typically 8 bytes.***

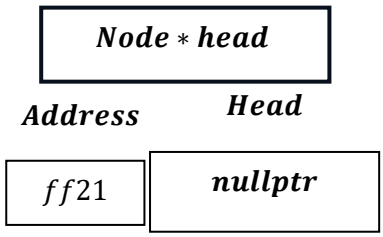
→ ***prev(a pointer to previous Node) is typically 8 bytes.***

→ ***Therefore , the size of the Node structure is typically 20 bytes(4 bytes for data + 8 bytes for next + 8 bytes for previous) .***

Say,



1st condition : HEAD POINTER points NULLPTR(LIST IS EMPTY)



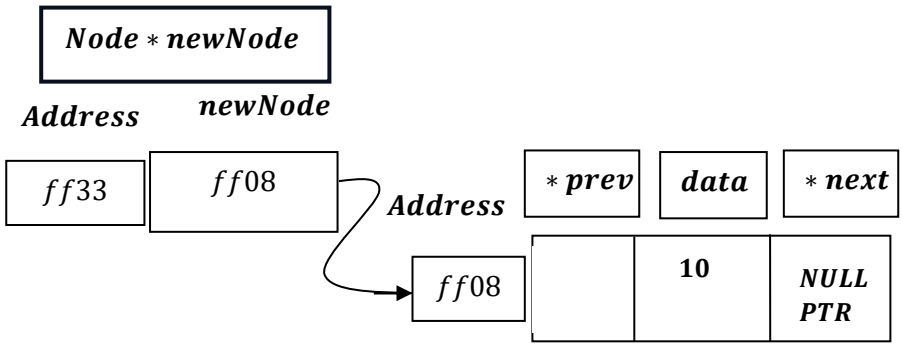
Then:

```
        case 2:
            cout << "Enter data to insert: ";
            cin >> data;
            insertAtEnd(data);
            break;
```

And, say data is : 10.

```
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = nullptr;
```

i.e.,

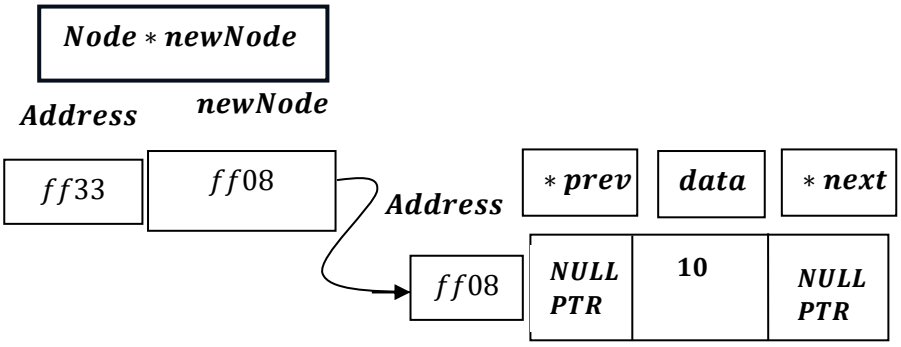


Now,

```
if (head == nullptr)
    {
        newNode->prev = nullptr;
        head = newNode;
        cout << data << " inserted at the end." << endl;
        return;
    }
```

i.e.,

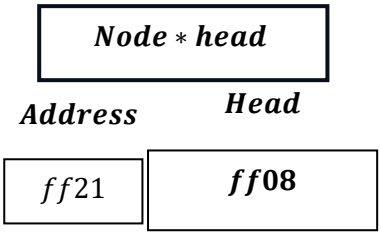
```
        newNode->prev = nullptr;
```



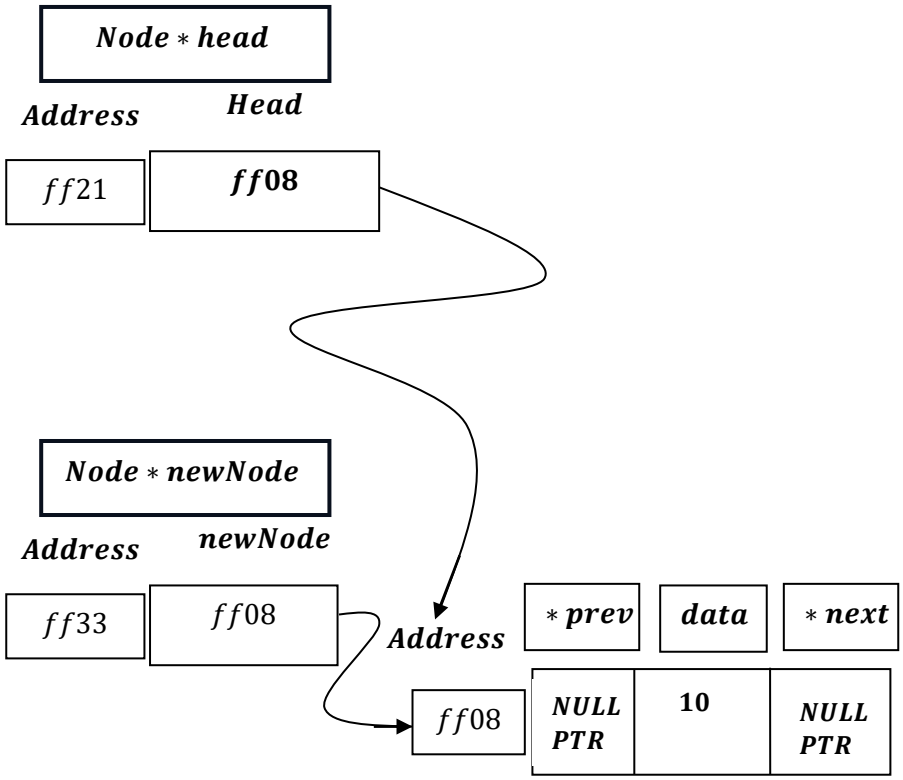
Next,

```
        head = newNode;
```

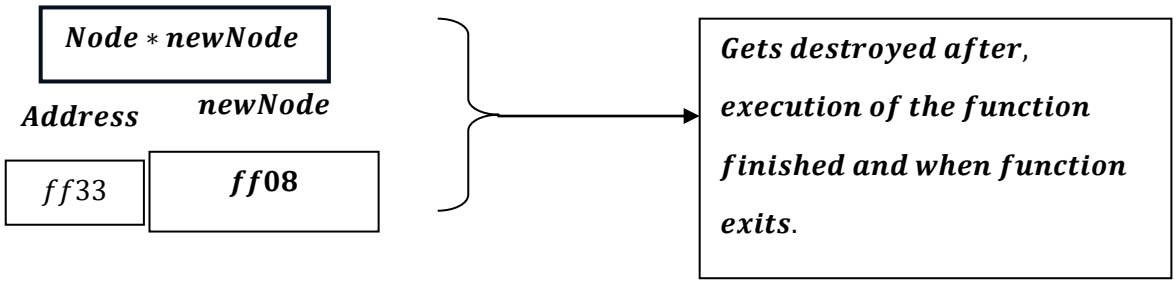
$i.e, head = ff08$



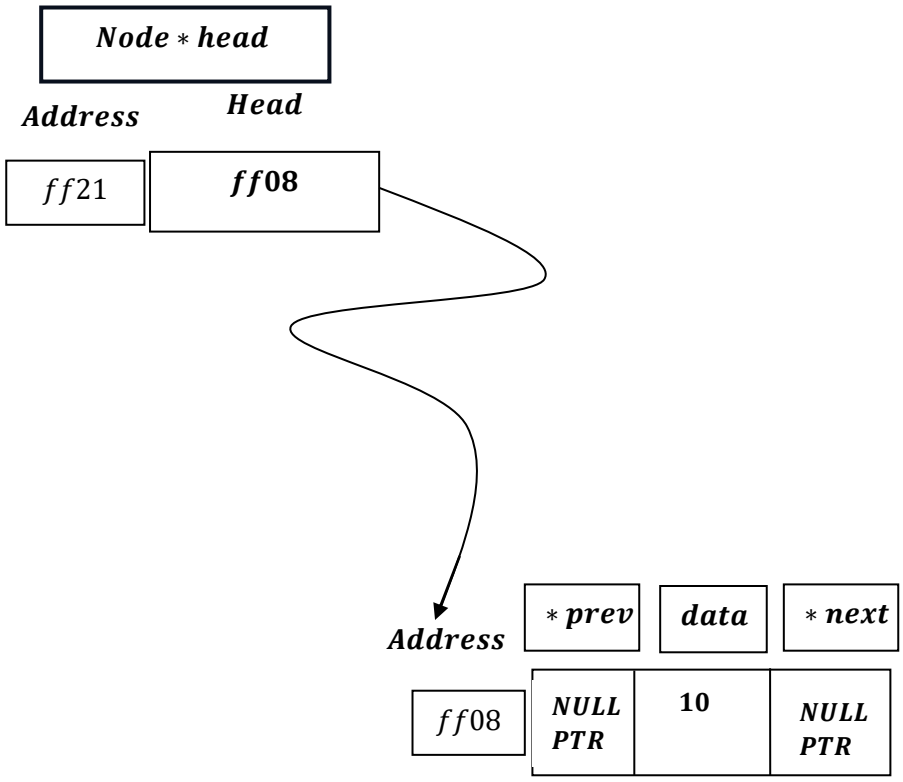
Hence , head pointer will now point to address : ff08.



Now, Node * newNode temporary variable, therefore gets destroyed after function execution get finished .



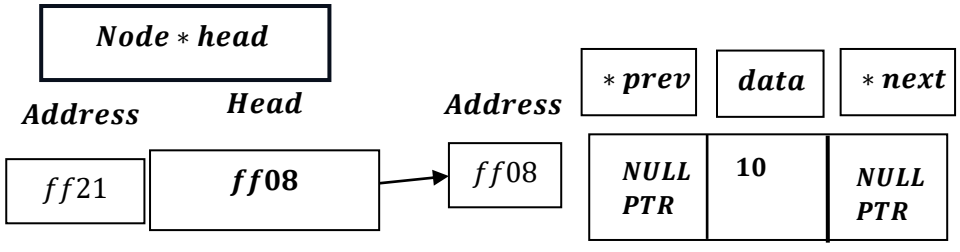
Hence , we are left with :



2nd condition

: HEAD POINTER not points to NULLPTR(LIST IS NOT EMPTY)

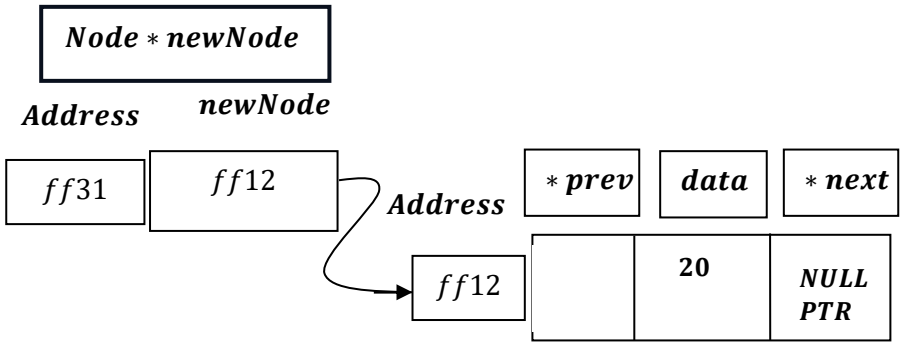
We have:



Lets insert another data , say **20** .

```
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = nullptr;
```

$i.e.,1.Creation\ of\ newNode\ pointer\ and\ 2.assigining\ data\ and\ next\ pointer\ in\ memory\ allocation, pointed\ by\ newNode\ pointer.$




```
if (head == nullptr)
```

$here, Head \neq NULLPTR, hence\ false, therefore\ exit\ if.$

```
    Node *temp = head;
```

$can\ be\ broken\ as, Node * temp;$

$temp = head;$

The Breakdown

1. **`Node *temp = head;` (Declaration with Initialization)**

   This single line performs two actions at once:

   - **Declaration**: `Node *temp` tells the compiler to create a new variable named `temp` that is a pointer to a `Node` object.
   - **Initialization**: `= head` immediately assigns the memory address currently stored in the `head` pointer to the new `temp` pointer.

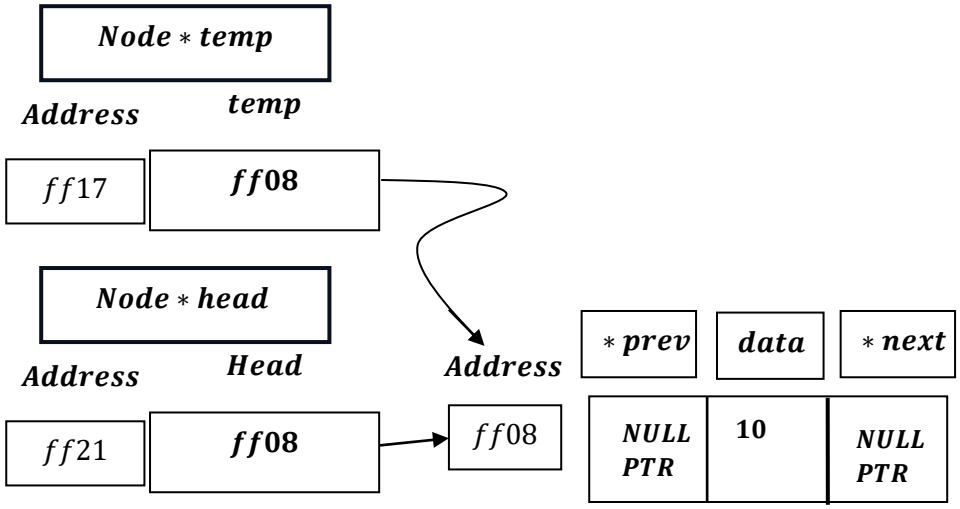
This is the most common and preferred way to write it because it's concise and ensures the pointer holds a valid (or `nullptr`) value from the moment it's created.

2. **`Node *temp;` followed by `temp = head;` (Separate Declaration and Assignment)**

   This is the two-step version of the same process:

   - **Declaration**: `Node *temp;` creates the pointer variable `temp`. At this exact moment, `temp` is **uninitialized** and holds a garbage memory address.
   - **Assignment**: `temp = head;` then copies the memory address from `head` into the `temp` pointer.

i.e., temp = head = ff08. hence temp will now point to address → ff08.



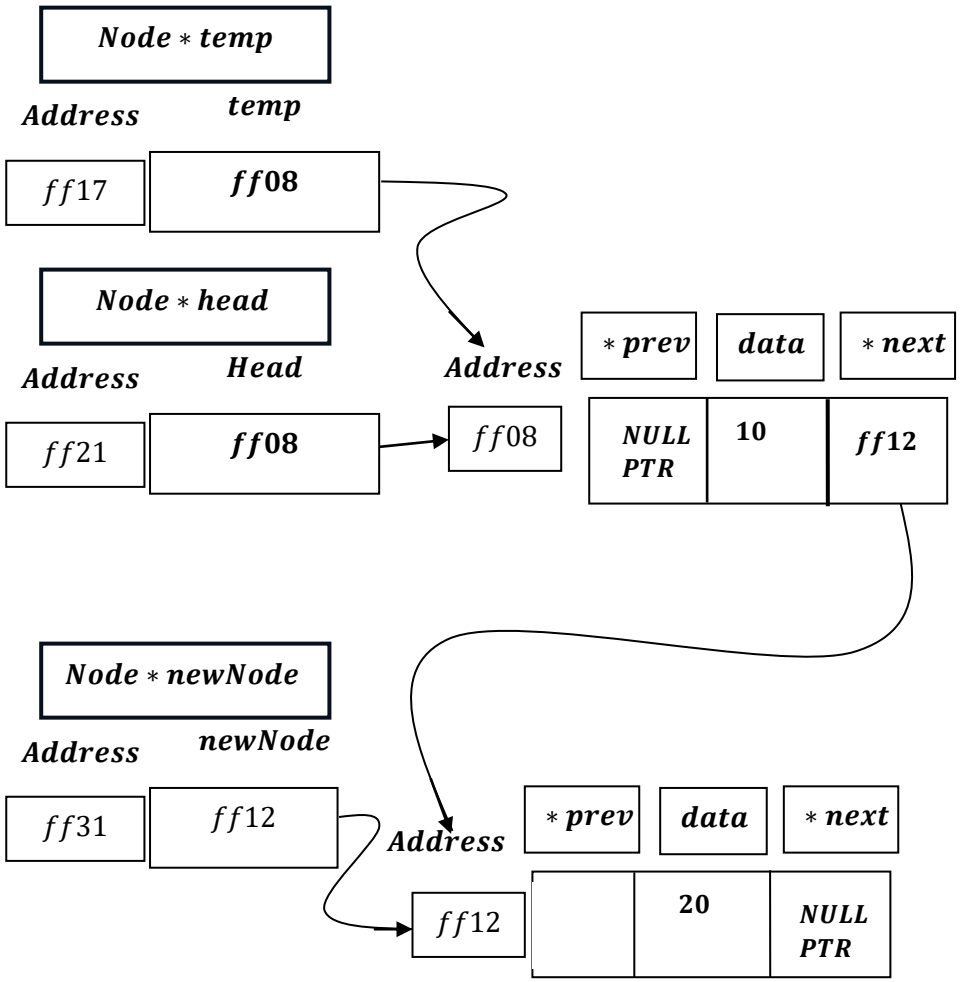
Next,

```
while (temp->next != nullptr)
{
    temp = temp->next;
}
```

Here , temp → next = nullptr , hence while loop will not execute.

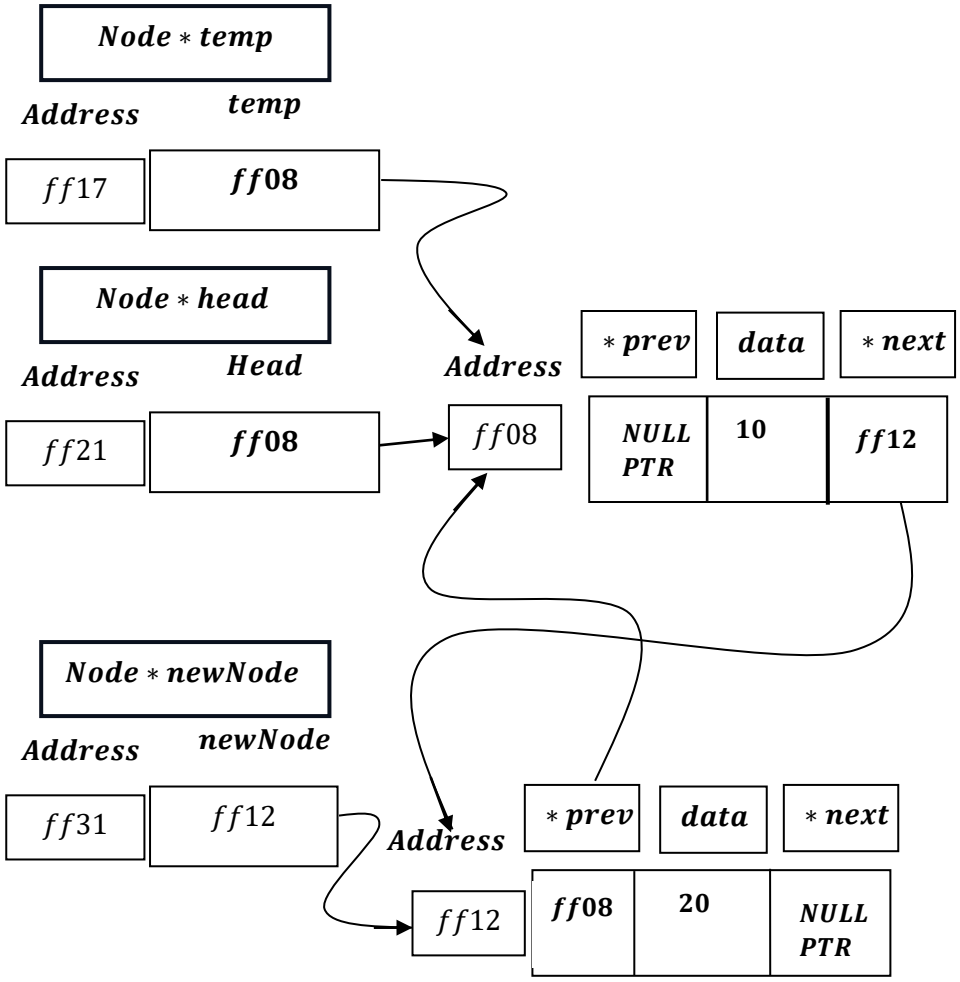
```
    temp->next = newNode;
```

$i.e. temp \rightarrow next = newNode = ff12.$

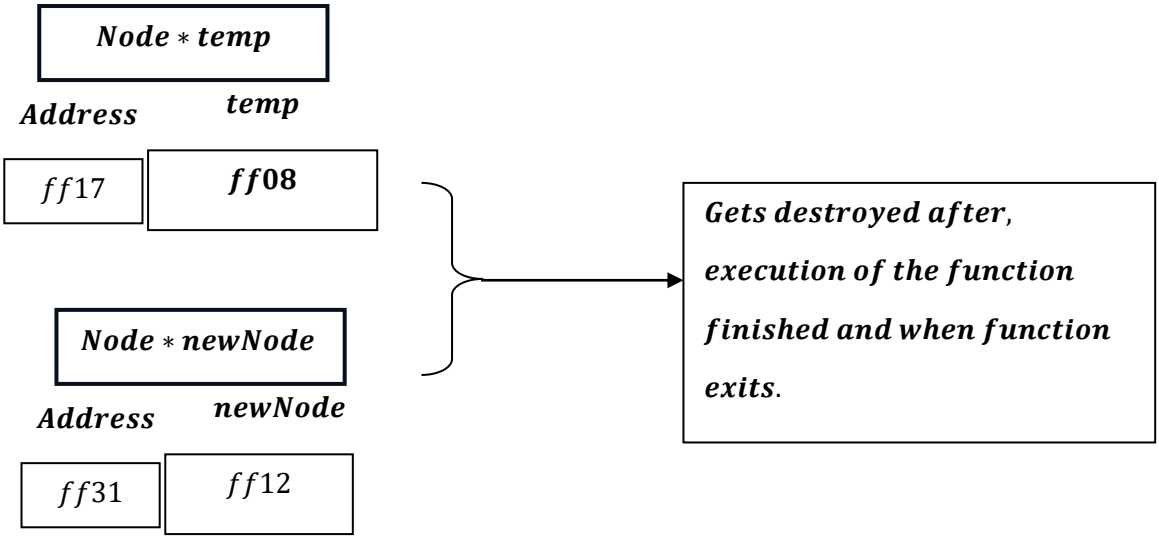



```
    newNode->prev = temp;
```

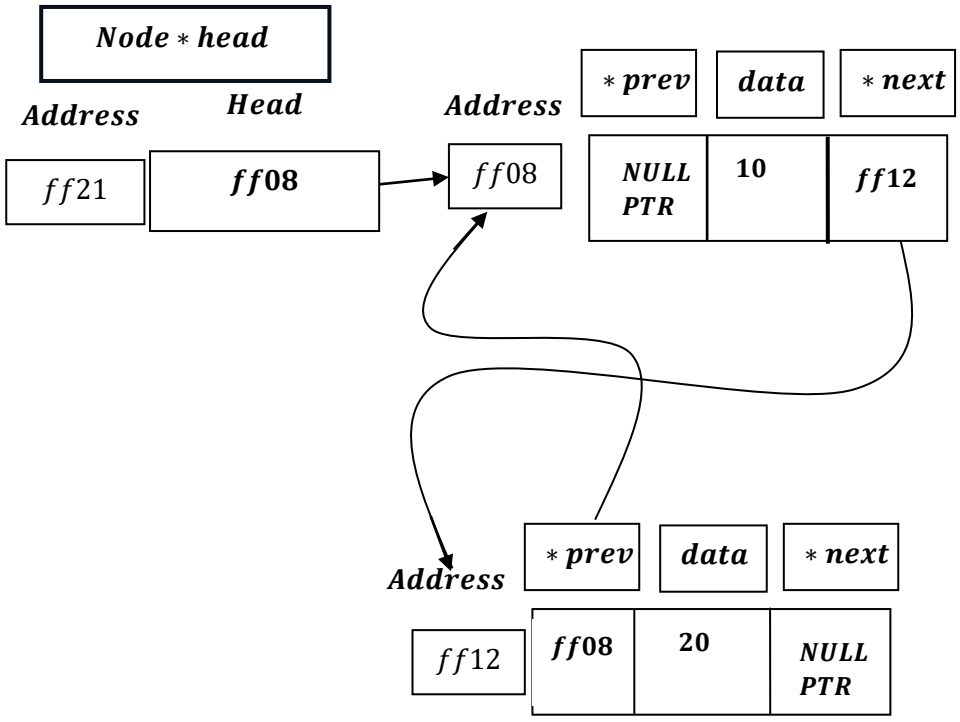
$i.e., newNode \rightarrow prev = temp = ff08.$



Now, ***Node * newNode , Node * temp temporary variables, therefore gets destroyed after function execution get finished .***



Hence , we are left with :



Doubly Linked List - Insert At End [Working Summary]:

First, a new node is created, its data is set, and its `next` pointer is assigned `nullptr` since it will always be the new last node.

→ **If the `head` pointer is null (i.e., the list is empty)**, the new node's `prev` pointer is also set to `nullptr`. The `head` pointer is then updated to point to this new node, making it the only element in the list.

→ **If the list is not empty**, a temporary pointer (`temp`) traverses the list from the `head` until it reaches the current last node (the node whose `next` pointer is `nullptr`). Once at the end, the original last node's (`temp`) `next` pointer is set to point to the `newNode`, and the `newNode`'s `prev` pointer is set to point back to `temp`, officially attaching the new node at the end of the list.

Time Complexity of the program

```
void insertAtEnd(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));

    newNode->data = data;

    newNode->next = nullptr;

    if (head == nullptr)
    {

        newNode->prev = nullptr;

        head = newNode;

        cout << data << " inserted at the end." << endl;

        return;
    }

    Node *temp = head;


    while (temp->next != nullptr)
    {
        temp = temp->next;
    }

    temp->next = newNode;

    newNode->prev = temp;
    cout << data << " inserted at the end." << endl;
}
```

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$runs` n-1` times, when\ head \neq NULL$

$statement\ also\ runs` n-1` times.$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

Best Case: $O(1)$, when head is Null → Absolute Best Case

→ The function begins by allocating memory for a new node using malloc. This operation takes constant time, typically $O(1)$, regardless of the size of the list.

→ The memory allocated for integer data pointed by pointer newNode is assigned to data input by user takes constant time, typically $O(1)$.

→ The memory allocated for pointer variable next pointed by pointer newNode is assigned to nullptr takes constant time, typically $O(1)$.

→ The function checks if the head of the list is null, indicating an empty list. If the list is empty, the new node becomes the head. This step takes constant time, $O(1)$.

- prev pointer of allocated memory pointed by newNode pointer set to `nullptr` , takes constant amount of time $O(1)$.
- newNode pointer's value assigned to head pointer , takes constant amount of time $O(1)$.
- Output to user , the integer `data` input with "inserted at the end" statement , which is a string , takes constant amount of time $O(1)$.
- Then , return statement runs, takes constant amount of time $O(1)$ and exit from the function.

Hence Total Time Complexity: $O(1) + O(1) + O(1) + \big(O(1) + (O(1) + O(1) + O(1))\big) = O(1)$

Best Case → $O(1)$, when 2nd node is inserted i.e. if `n = 1`, where n is node and another node is Inserted . → Not the absolute best case.

→ The function begins by allocating memory for a new node using malloc. This operation takes constant time, typically $O(1)$, regardless of the size of the list.

→ The memory allocated for integer data pointed by pointer newNode is assigned to data input by user takes constant time, typically $O(1)$.

→ The memory allocated for pointer variable next pointed by pointer newNode is assigned to nullptr takes constant time, typically $O(1)$.

→ The function checks if the head of the list is null, now list is having a single node , whose prev pointer is NULLPTR , next pointer is also NULLPTR, and with a single integer data , hence the check returns false , hence, takes constant time, typically $O(1)$.

→ Now temp pointer of structure Node assigned to the value of head pointer, makes temp pointer points to the address that head pointer points to , this process takes constant time, typically $O(1)$.

→ while $(temp \rightarrow next \neq NULLPTR)$, but $temp \rightarrow next = NULLPTR$, while loop condition fails here and hence fails to run, this condition checking takes takes constant amount of time $O(1)$.

→ next pointer pointed by temp pointer assigned to value of newNode pointer of Node structure , hence next pointer will now point to the address to which newNode pointer pointing to , this process takes constant time, typically $O(1)$.

$\rightarrow$ ***prev pointer pointed by newNode pointer of Node structure assigned to the value of temp pointer of Node structure , hence prev pointer will now point to the address to which temp pointer pointing to ,***

this process takes constant time, typically $O(1)$.

$\rightarrow$ ***Output to user , the integer `data` input with "inserted at the end" statement , which is a string , takes constant amount of time*** $O(1)$.

Hence Total Time Complexity: $O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1)$.

Why not Absolute Best Case ?

$\Longrightarrow$ ***When*** $n = 1$, ***loop runs*** 0 ***times***. [***maintaining*** , $n - 1$ ***times***]
$\Longrightarrow$ ***When*** $n = 2$, ***loop runs*** 1 ***times***. [***maintaining*** , $n - 1$ ***times***]
$\Longrightarrow$ ***When*** $n = 3$, ***loop runs*** 2 ***times***. [***maintaining*** , $n - 1$ ***times***]
.
.
.
$\Longrightarrow$ ***When*** $n = n$, ***loop runs*** $n - 1$ ***times***. [***maintaining*** , $n - 1$ ***times***]

Hence , when $n \neq 0$ ***i.e. List is not empty , it maintains*** $n - 1$ ***times running of loop , therefore , when*** $n = 1$ ***, loop doesnot runs , hence function remains at constant time complexity***.

When $n > 1$ ***, Loop runs , we cannot take this scenario as function remaining at constant time , therefore when*** $n = 1$ ***, as function remains at constant time , we can take this as best case but not the `Absolute Best Case`. The `Absolute Best Case ` is when List is Empty***.

$\underline{Worst\ Case: O(n), when\ n > 1}$

→ *The function begins by allocating memory for a new node using malloc. This operation takes constant time, typically* $O(1)$, *regardless of the size of the list.*

→ *The memory allocated for integer data pointed by pointer newNode is assigned to data input by user takes constant time, typically* $O(1)$.

→ *The memory allocated for pointer variable next pointed by pointer newNode is assigned to nullptr takes constant time, typically* $O(1)$.

→ *The function checks if the head of the list is null, now list is having a single node , whose prev pointer is NULLPTR , next pointer is also NULLPTR, and with a single integer data , hence the check returns false , hence, takes constant time, typically* $O(1)$.

→ *Now temp pointer of structure Node assigned to the value of head pointer, makes temp pointer points to the address that head pointer points to , this process takes constant time, typically* $O(1)$.

→ *while* $(temp \rightarrow next \neq NULLPTR)$, *and now , temp* $\rightarrow next \neq NULLPTR$, *meets the condition of while loop, hence body of the while loop i.e.*

temp pointer of Node structure gets assigned to the value $(Address)$*held by next pointer of allocated memory pointed by temp pointer , i.e. temp will now point to next node and traverse to* $n-1$ *node of list containing `n` nodes .*

$-$***Which makes time complexity here is not constant i.e.*** $O(n-1) = O(n)$.

→ ***next pointer pointed by temp pointer assigned to value of newNode pointer of Node structure , hence next pointer will now point to the address to which newNode pointer pointing to , this process takes constant time, typically*** $O(1)$.

→ ***prev pointer pointed by newNode pointer of Node structure assigned to the value of temp pointer of Node structure , hence prev pointer will now point to the address to which temp pointer pointing to , this process takes constant time, typically*** $O(1)$.

→ ***Output to user , the integer `data` input with "inserted at the end" statement , which is a string , takes constant amount of time*** $O(1)$.

Hence Total Time Complexity: $O(1) + O(1) + O(1) + O(1) + O(1) + O(n) + O(1) + O(1) + O(1) = O(n)$.

Doubly Linked List	***Condition***	***Time Complexity***
InsertAtEnd	1. ***List is Empty***	$O(1)$
	2. ***List is not Empty***	$O(n)$

We can say , if List is Empty , we get our best case scenario, and when list is not empty , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine where the insertion to be made , hence we can say average case is our worst case here i. e. always the entire list is traversed i.e. $O(n) = \Theta(n)$.

In the case of insertAtEnd, the insertion always happens at the end, regardless of the current state of the list. This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (best case). If the list is not empty, the execution time is proportional to the length of the list (worst case). Since the insertion position is predetermined, there's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence insertAtEnd doesn't have a true average case.

Note , by relation: $\Omega \leq \Theta \leq O$ *, we get:*

$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$

Doubly Linked List	***Cases***	***Time Complexity***
InsertAt End	**1.** ***Best Case***	$\boldsymbol{\Omega(1)}$
	2. ***Worst Case***	$\boldsymbol{O(n)}$
	3. ***Average Case***	$\boldsymbol{\Theta(n)}$

****